Cloud Networking

AWS Running Containers on a Managed Service

Appunti spiegati passo passo

Fonte usata

PDF: “16 - CN - Running Containers on a Managed Service”. Il documento presenta il Lab 8.2: esecuzione di container su un servizio gestito, usando AWS Elastic Beanstalk per il sito applicativo e Amazon Aurora Serverless per sostituire il database MySQL containerizzato.

Obiettivo degli appunti

Capire il passaggio dal semplice uso di container su un host EC2 a un’architettura piu gestita, scalabile e con minore overhead operativo: immagini container in Amazon ECR, deployment dell’applicazione con Elastic Beanstalk, database gestito con Aurora Serverless e integrazione tramite API Gateway proxy.

Indice degli argomenti

- 1. Idea generale del laboratorio
- 2. Dal Lab 8.1 al Lab 8.2: cosa cambia
- 3. Concetti chiave: managed service, ECR, Elastic Beanstalk, Aurora Serverless
- 4. Scenario iniziale
- 5. Scenario finale
- 6. Svolgimento passo passo delle task
- 7. Componenti -> Analisi del flusso -> Criticita -> Raccomandazioni
- 8. Tabelle di confronto utili per l’esame
- 9. Domande d’esame con risposta subito sotto
- 10. Glossario rapido

1. Idea generale del laboratorio

Il laboratorio mostra l'evoluzione naturale dopo la containerizzazione. Nel Lab 8.1 l'applicazione era stata migrata in container Docker: la web application e il database MySQL potevano essere eseguiti come container. Nel Lab 8.2 il passo successivo e ridurre il carico di gestione: l'applicazione resta containerizzata, ma viene eseguita su un servizio gestito; il database non viene piu mantenuto come container MySQL, ma spostato su Amazon Aurora Serverless.

Idea chiave

Containerizzare non significa necessariamente gestire tutto a mano. In ambiente cloud e spesso piu corretto usare servizi gestiti: Elastic Beanstalk gestisce il deployment e l'ambiente applicativo, Aurora Serverless gestisce il database relazionale e la sua capacita, ECR conserva le immagini container.

Prima	Dopo	Motivazione
Container eseguiti manualmente su un host EC2	Applicazione distribuita tramite AWS Elastic Beanstalk	Meno gestione manuale dell'ambiente, provisioning piu rapido, integrazione con load balancing e scaling.
Database MySQL in container	Amazon Aurora Serverless	Il database diventa un servizio gestito: scaling automatico, minore manutenzione, maggiore disponibilita.
Immagini locali	Immagini in Amazon ECR	Le immagini sono archiviate in un registry gestito e riusabili da altri servizi AWS.
Endpoint applicativo diretto	API Gateway proxy verso il backend	L'accesso applicativo puo essere astratto, esposto e controllato tramite API Gateway.

2. Dal Lab 8.1 al Lab 8.2: cosa cambia

Il Lab 8.1 si concentra sulla migrazione a Docker; il Lab 8.2 si concentra sulla gestione cloud-native dei container e del database. La differenza non e solo tecnica, ma architetturale: si passa da un modello "self-managed" a un modello "managed service".

Aspetto	Lab 8.1 - Docker su host EC2	Lab 8.2 - Managed services
Gestione runtime	Docker Engine sull'istanza EC2 usata come host.	Elastic Beanstalk crea e gestisce l'ambiente applicativo.
Database	MySQL containerizzato, da configurare e mantenere.	Aurora Serverless, database relazionale gestito e scalabile.
Scalabilita	Dipende dalla configurazione manuale dei container/host.	Piu integrata: Beanstalk puo distribuire istanze, bilanciamento e deployment.
Persistenza dati	Richiede volumi e backup gestiti correttamente dall'utente.	Gestita dal servizio database RDS/Aurora.
Responsabilita operativa	Maggiore responsabilita del team: patch, runtime, container, DB.	Responsabilita ridotta sul DB e sull'ambiente applicativo.

Da esame

La scelta di un servizio gestito e coerente con il modello cloud perche permette di delegare al provider parte della gestione dell'infrastruttura. Il consumer rimane responsabile di configurazione, codice, dati e sicurezza applicativa, ma non deve amministrare direttamente tutto lo stack sottostante.

3. Concetti chiave

Managed service

Un servizio gestito e un servizio cloud in cui il provider si occupa di provisioning, manutenzione, disponibilita, patching di parte dell'infrastruttura e, in molti casi, scaling. Il vantaggio principale e ridurre l'overhead operativo.

Amazon ECR

Amazon Elastic Container Registry e il registry gestito in cui vengono caricate le immagini container. Le immagini diventano recuperabili da servizi come Elastic Beanstalk, ECS, EKS o pipeline CI/CD.

AWS Elastic Beanstalk

Servizio PaaS che consente di distribuire applicazioni web concentrandosi sul codice e sulla configurazione applicativa. Nel lab viene usato per deployare il sito applicativo containerizzato.

Amazon Aurora Serverless

Configurazione on-demand di Amazon Aurora, compatibile con MySQL e PostgreSQL. Scala automaticamente in base al traffico e puo ridurre i costi quando il database non e usato.

API Gateway proxy

API Gateway puo esporre un endpoint pubblico e inoltrare le richieste verso il backend applicativo. In questo modo il client non deve conoscere direttamente l'endpoint interno del servizio.

Security group e subnet

Il corretto funzionamento dipende dalla rete: Elastic Beanstalk, EC2 e Aurora devono essere posizionati in subnet coerenti e avere regole di sicurezza che permettono il traffico necessario, ad esempio dalla web app verso il database.

Servizio	Tipo	Ruolo nel laboratorio	Perche e importante
Elastic Beanstalk	PaaS / servizio gestito applicativo	Esegue la web application containerizzata.	Riduce la gestione manuale dell'ambiente e facilita deployment/scaling.
Aurora Serverless	Database relazionale gestito	Sostituisce il container MySQL.	Scala in base al carico e gestisce disponibilita e manutenzione del DB.
Amazon ECR	Container registry gestito	Conserva l'immagine del sito/applicazione.	Permette a Beanstalk e altri servizi di recuperare immagini versionate.
API Gateway	Gateway/API proxy	Instrada richieste verso il backend.	Astrazione, controllo e pubblicazione ordinata dell'API.
VPC/Subnet/Security Group	Networking e sicurezza	Consentono comunicazione tra app, DB e servizi.	Una configurazione errata blocca la connessione anche se app e DB sono corretti.

4. Scenario iniziale

All'inizio del Lab 8.2 esiste una VPC con una public subnet e un'istanza IDE su EC2. Sono presenti immagini container in Amazon ECR: una immagine per il website e una immagine per il database. Tuttavia, l'obiettivo e non mantenere il database come container, ma spostarlo su Aurora Serverless.

Lab: Starting Scenario



Figura 1 - Scenario iniziale del Lab 8.2: ECR contiene le immagini, l'IDE gira su EC2 dentro una public subnet, ma l'applicazione non e ancora distribuita tramite servizio gestito e il database deve essere migrato verso Aurora Serverless.

Analisi dello scenario iniziale

Componenti presenti: Region, VPC, public subnet, IDE su EC2, Amazon ECR con immagine website e immagine database. Criticita principale: la sola presenza di immagini container non basta a ottenere scalabilita e gestione automatizzata; serve un servizio applicativo gestito e un database gestito.

Elemento iniziale	Funzione	Limite da superare
IDE su EC2	Ambiente di lavoro e controllo da cui eseguire comandi e configurazioni.	Non deve diventare il punto operativo definitivo dell'applicazione.
ECR - Website image	Contiene l'immagine container della web application.	Deve essere usata da un ambiente di deployment, non solo conservata.
ECR - Database image	Rappresenta la vecchia opzione di database containerizzato.	Nel Lab 8.2 viene superata da Aurora Serverless.
Public subnet 1	Sottorete usata per risorse accessibili/gestibili.	Serve configurare correttamente subnets e security group per RDS/Beanstalk.

5. Scenario finale

Nello scenario finale l'applicazione viene deployata in un ambiente Elastic Beanstalk. Il traffico entra attraverso un Data API/API Gateway proxy, passa verso il deployment applicativo e raggiunge istanze EC2 che ospitano

l'immagine del website. Il database e fornito da Aurora Serverless, distribuito su piu subnet, con comunicazione dalla web application verso il servizio database.

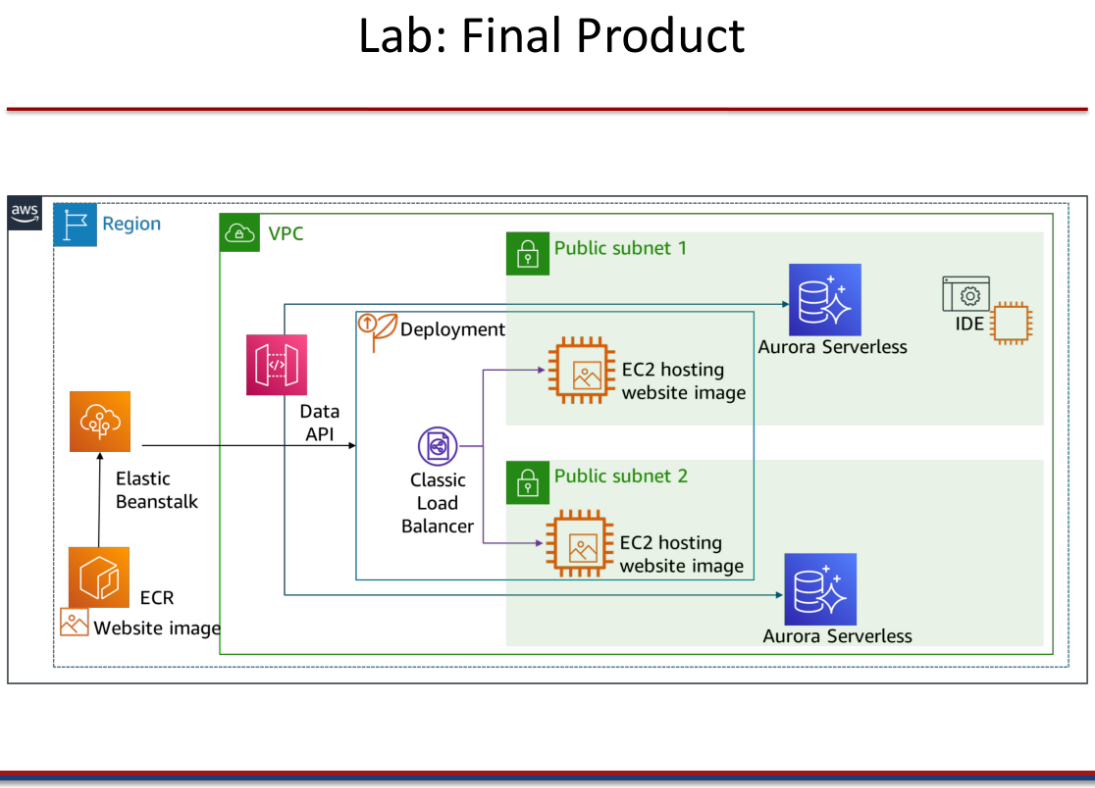


Figura 2 - Scenario finale del Lab 8.2: Elastic Beanstalk usa l'immagine del website da ECR, il deployment usa EC2 e load balancing, mentre Aurora Serverless sostituisce il database MySQL containerizzato.

Flusso finale

Utente/API client -> API Gateway proxy/Data API -> ambiente Elastic Beanstalk -> Classic Load Balancer -> istanze EC2 che eseguono la website image -> connessione verso Aurora Serverless -> risposta del database -> ritorno verso il client.

Componente finale	Ruolo	Problema che risolve
Elastic Beanstalk	Crea e gestisce l'ambiente applicativo che esegue il container del website.	Riduce l'effort di deploy e gestione rispetto all'esecuzione manuale su EC2.
Classic Load Balancer	Distribuisce il traffico tra le istanze EC2 del deployment.	Evita che una singola istanza diventi collo di bottiglia.
EC2 hosting website image	Esegue il container applicativo partendo dall'immagine caricata in ECR.	Rende il deployment riproducibile e integrato nel servizio gestito.
Aurora Serverless	Database relazionale gestito e scalabile.	Sostituisce il DB containerizzato, riducendo gestione manuale e migliorando elasticita.
Public subnet 1 e 2	Distribuiscono risorse su piu zone/subnet.	Supportano maggiore resilienza e configurazioni multi-AZ.
API Gateway proxy/Data API	Espone e instrada le richieste verso il backend.	Fornisce astrazione dell'endpoint e controllo dell'accesso all'applicazione.

6. Svolgimento passo passo delle task

1. Preparare l'ambiente di sviluppo

Verificare che l'IDE su EC2 sia operativo, che AWS CLI funzioni, che Docker/ECR siano accessibili e che siano disponibili i file e gli script del laboratorio. Questo passaggio evita errori successivi su permessi, credenziali o repository mancanti.

2. Configurare le subnet per Amazon RDS ed Elastic Beanstalk

RDS/Aurora e Beanstalk devono usare subnet compatibili. Aurora Serverless richiede un DB subnet group composto da subnet adeguate; Elastic Beanstalk deve poter eseguire l'applicazione e raggiungere il database. La rete va progettata prima del deployment.

3. Configurare Aurora Serverless

Si crea un cluster Aurora Serverless compatibile con MySQL. Si definiscono database name, username, password, VPC, subnet group, security group e capacita di scaling. Aurora Serverless scala in base al traffico e puo sospendere/ridurre capacita quando non serve.

4. Revisionare l'immagine container

Prima di deployare l'applicazione, si controlla che l'immagine website sia corretta: deve includere codice, dipendenze, comando di avvio e porta applicativa. L'immagine deve essere disponibile in ECR.

5. Configurare la comunicazione container-database

L'applicazione deve ricevere endpoint, porta, nome database, utente e password di Aurora. In genere questi parametri sono passati come variabili d'ambiente o configurazione dell'ambiente Beanstalk.

6. Creare gli oggetti del database applicativo

Il database deve contenere schema, tabelle e utente applicativo attesi dal codice. In questo caso l'applicazione dei fornitori richiede oggetti come database, utente e tabelle per i dati supplier.

7. Caricare i dati dei fornitori

Si popola il database con i dati iniziali. Questo passaggio consente all'applicazione di restituire risultati reali quando il frontend o l'API interrogano il backend.

8. Revisionare policy e ruolo IAM per Elastic Beanstalk

Elastic Beanstalk necessita dei permessi corretti per creare/gestire risorse, accedere a ECR e orchestrare l'ambiente. IAM role e policy devono rispettare il principio del minimo privilegio, ma includere le autorizzazioni necessarie.

9. Creare l'applicazione Elastic Beanstalk

Si crea l'applicazione e l'ambiente Beanstalk usando l'immagine container. Beanstalk si occupa di deployment, istanze EC2, bilanciamento e configurazioni operative secondo il profilo scelto.

10. Configurare API Gateway proxy

API Gateway viene usato per esporre una API che inoltra le richieste al backend Beanstalk. Questo permette di pubblicare un endpoint controllato, separando il client dall'infrastruttura applicativa sottostante.

7. Componenti -> Analisi del flusso -> Criticita -> Raccomandazioni

Componenti

- Amazon ECR: repository delle immagini container.
- AWS Elastic Beanstalk: servizio gestito per deploy e gestione della web application.
- EC2 nel deployment: istanze sottostanti che eseguono il container della website image.
- Classic Load Balancer: distribuisce il traffico tra piu istanze applicative.
- Amazon Aurora Serverless: database relazionale gestito, compatibile MySQL/PostgreSQL.
- VPC, public subnets e security groups: rete logica e controlli di accesso.
- API Gateway proxy/Data API: livello di esposizione e inoltro delle richieste verso il backend.

Analisi del flusso

1. Il client invia una richiesta all'endpoint API esposto.
2. API Gateway riceve la richiesta e la inoltra verso l'applicazione ospitata in Elastic Beanstalk.
3. Il load balancer distribuisce la richiesta verso una delle istanze EC2 del deployment.
4. L'istanza esegue il container della web application ottenuto da ECR.
5. L'applicazione interroga Aurora Serverless usando endpoint, credenziali e security group corretti.
6. Aurora restituisce i dati; la risposta risale verso l'applicazione, il load balancer, API Gateway e infine il client.

Identificazione criticita

Criticita	Causa tipica	Conseguenza
Connessione al DB fallita	Security group non consente traffico dalla web app verso Aurora sulla porta corretta.	L'applicazione risponde con errore o non carica dati.
Errore di autenticazione DB	Username/password/database name non coerenti con quelli creati in Aurora.	Query non eseguibili.
Immagine non scaricabile	Permessi IAM o URI ECR non corretti.	Elastic Beanstalk non riesce ad avviare il container.
Ambiente Beanstalk non sano	Porta applicativa non coerente, health check errato o container non avviato.	Deployment in stato degraded/unhealthy.
API Gateway proxy non raggiunge backend	URL, stage, integration o routing errato.	Client riceve 4xx/5xx o timeout.

Raccomandazioni

- Separare il piano applicativo dal piano dati: applicazione in Beanstalk, database in Aurora Serverless.
- Usare security group specifici: uno per l'applicazione e uno per il database; il DB deve accettare traffico solo dal security group applicativo.
- Conservare le immagini in ECR con tag chiari, ad esempio versioni applicative progressive.
- Non salvare credenziali nel codice: usare variabili d'ambiente, Secrets Manager o parametri gestiti quando disponibili.
- Monitorare health check, log applicativi, metriche di Beanstalk e metriche Aurora per individuare errori di deploy o saturazioni.
- Usare API Gateway quando serve un layer di esposizione controllato, versionabile e separato dal backend.

8. Tabelle di confronto utili per l'esame

Criterio	MySQL in container	Aurora Serverless
Gestione	L'utente gestisce immagine, runtime, backup, aggiornamenti e persistenza.	AWS gestisce gran parte dell'infrastruttura database.
Scalabilita	Da progettare manualmente, spesso complessa per database stateful.	Scala automaticamente in base al traffico.
Disponibilita	Dipende da configurazione, volumi, backup e	Progettata come servizio RDS/Aurora gestito.

	host.	
Costo	Dipende da EC2/host sempre attivo e storage configurato.	On-demand/serverless: puo adattarsi all'uso.
Uso ideale	Test, sviluppo, ambienti temporanei, massima portabilita locale.	Produzione cloud, riduzione overhead, necessita di elasticita e gestione semplificata.

Criterio	Docker manuale su EC2	Elastic Beanstalk
Controllo	Molto alto: l'utente gestisce host, Docker e avvio container.	Medio: si configura l'ambiente ma molti dettagli sono gestiti dal servizio.
Overhead operativo	Alto.	Ridotto.
Scalabilita	Da implementare manualmente o con altri servizi.	Integrabile con load balancing e autoscaling.
Deployment	Basato su comandi manuali o script.	Gestito dal servizio, piu adatto a pipeline applicative.
Quando usarlo	Ambienti didattici, test controllati, massima personalizzazione.	Web application cloud che richiedono deploy rapido e gestione semplificata.

Criterio	Elastic Beanstalk	ECS/EKS/Fargate
Livello di astrazione	PaaS orientato ad applicazioni web.	Piattaforme/container orchestration piu orientate a microservizi e cluster.
Complessita	Piu semplice.	Maggiore, soprattutto EKS.
Controllo orchestrazione	Limitato rispetto a ECS/EKS.	Maggiore controllo su task, servizi, pod, cluster e scheduling.
Scenario tipico	Applicazione web monolitica o semplice container web.	Architetture a microservizi, orchestrazione avanzata, deployment multi-servizio.

9. Domande d'esame con risposta subito sotto

Domanda 1. Qual e l'obiettivo principale del Lab 8.2?

Risposta: Ridurre lo sforzo di gestione dell'applicazione containerizzata e migliorare la scalabilita, spostando il sito su AWS Elastic Beanstalk e il database su Amazon Aurora Serverless.

Domanda 2. Perche nel Lab 8.2 si sostituisce il database MySQL containerizzato con Aurora Serverless?

Risposta: Perche un database stateful richiede gestione di persistenza, backup, disponibilita e scaling. Aurora Serverless delega questi aspetti al servizio gestito e scala automaticamente in base al traffico.

Domanda 3. Che ruolo ha Amazon ECR nello scenario?

Risposta: ECR conserva le immagini container, in particolare la website image, permettendo ai servizi AWS di recuperarle in modo controllato e versionato.

Domanda 4. Che differenza c'e tra eseguire un container manualmente su EC2 e usare Elastic Beanstalk?

Risposta: Nel primo caso l'utente gestisce direttamente host, Docker, comandi di avvio, porte e aggiornamenti. Con Elastic Beanstalk il deployment e l'ambiente applicativo sono gestiti dal servizio, riducendo l'overhead operativo.

Domanda 5. Che cos'e Aurora Serverless?

Risposta: E una configurazione on-demand di Amazon Aurora, compatibile con MySQL e PostgreSQL, che scala automaticamente in base al traffico e puo ridurre capacita o fermarsi quando non viene usata.

Domanda 6. Quali componenti compaiono nello scenario finale del laboratorio?

Risposta: Elastic Beanstalk, Amazon ECR, API Gateway/Data API, deployment applicativo, Classic Load Balancer, istanze EC2 che ospitano la website image, public subnet e Aurora Serverless.

Domanda 7. Perche sono importanti subnet e security group nel laboratorio?

Risposta: Perche anche se applicazione e database sono configurati correttamente, la comunicazione fallisce se le subnet non sono compatibili o se i security group non permettono il traffico tra web application e database.

Domanda 8. Qual e il vantaggio del load balancer nello scenario finale?

Risposta: Distribuisce le richieste tra piu istanze applicative, riducendo il rischio di collo di bottiglia su una singola istanza e migliorando disponibilita e scalabilita.

Domanda 9. Qual e il ruolo di API Gateway proxy?

Risposta: Espone un endpoint API controllato e inoltra le richieste verso il backend, separando il client dall'infrastruttura interna dell'applicazione.

Domanda 10. Quale responsabilita rimane al cloud consumer anche usando servizi gestiti?

Risposta: Rimangono responsabilita su codice applicativo, configurazione, dati, credenziali, regole di sicurezza, parametri dell'ambiente e corretta integrazione tra servizi.

Domanda 11. Perche questo laboratorio rappresenta un passaggio verso un'architettura piu cloud-native?

Risposta: Perche non si limita a spostare software in container, ma usa servizi gestiti AWS per deployment, database, registry e accesso API, sfruttando elasticita e automazione del cloud.

Domanda 12. Quando sarebbe ancora sensato usare MySQL in container?

Risposta: In sviluppo, test, esercitazioni, ambienti temporanei o quando serve riprodurre rapidamente un database locale. Per produzione cloud, un servizio gestito come Aurora e generalmente piu robusto.

10. Glossario rapido

Termine	Definizione da ricordare
Managed service	Servizio in cui il provider gestisce parte rilevante dell'infrastruttura e dell'operativita.
ECR	Registry gestito AWS per immagini container.
Elastic Beanstalk	Servizio PaaS per distribuire e gestire applicazioni web.
Aurora Serverless	Database relazionale gestito che scala automaticamente in base alla domanda.
RDS	Servizio AWS per database relazionali gestiti; Aurora fa parte dell'ecosistema RDS.
Security group	Firewall virtuale a livello di istanza/risorsa che controlla traffico inbound e outbound.
Subnet group RDS	Gruppo di subnet usato da RDS/Aurora per posizionare il database nella VPC.
Load balancer	Componente che distribuisce traffico su piu istanze applicative.
API Gateway	Servizio per creare, pubblicare, proteggere e instradare API.
Website image	Immagine container della web application usata per il deployment.